🔧 TECHNICAL QUESTIONS

Q1. Walk me through how you would investigate a customer-reported issue where a transaction failed.

I'd follow a structured RCA approach. First, I'd reproduce the issue in a test/staging environment using the customer's transaction data. Then I'd parse the relevant logs — using Linux commands like grep, awk, or tail — to isolate the failure point. I'd check the payload (XML/CSV/JSON) for malformed fields, encoding issues, or missing values. Once the root cause is identified, I'd document the steps, verify the fix with the engineering team, and update the run-book so the same issue is resolved faster next time. I did exactly this during my Cognifyz internship — traced backend API failures through log analysis and reduced resolution time significantly.

Q2. How comfortable are you with Linux/UNIX command line?

Very comfortable for support tasks. I regularly use commands like grep, awk, sed, tail -f, find, ps, top, chmod, cron, and scp/sftp for log parsing, process monitoring, file management, and automation. I have an LPI Linux Essentials certification and have used Linux as my primary dev environment for all projects and internships.

Q3. You receive a malformed XML payload from a customer. How do you handle it?

First, I'd validate the XML structure using a parser (Python's xml.etree.ElementTree or lxml) to identify exactly which tag or attribute is malformed. I'd check for common issues — unclosed tags, special characters not escaped, wrong encoding (UTF-8 vs UTF-16), or schema violations. Then I'd write a quick Python script to clean or flag the problematic records, document the exact error with line references, and communicate the fix clearly to the customer with an example of the corrected structure.

Q4. Write a quick Python/Bash script to parse a log file and extract all ERROR lines.

Python:

python

```
with open("app.log") as f:
    errors = [line for line in f if "ERROR" in line]
for e in errors:
    print(e.strip())
```

Bash:

bash

```
grep "ERROR" app.log
# With timestamp filter:
```

grep "ERROR" app.log | awk '{print $1, $2, $NF}'

In my log analysis project I extended this to count error frequency by type, generate summary CSVs, and trigger alerts — which is directly applicable to TraceLink's monitoring workflows.

Q5. What is SFTP and how is it different from FTP?

SFTP (SSH File Transfer Protocol) transfers files over an encrypted SSH connection, making it secure. FTP transfers data in plaintext — no encryption — making it vulnerable to interception. For B2B integrations like TraceLink's, SFTP is the standard because it ensures secure, authenticated file transfers between trading partners. I've used SFTP in my automation scripts for scheduled file pulls and transfers via cron jobs.

Q6. What is AS2 and where is it used?

AS2 (Applicability Statement 2) is a protocol used for secure, real-time B2B data exchange over HTTP/HTTPS. It uses digital certificates and encryption (S/MIME) to ensure data integrity and non-repudiation. It's widely used in supply chains — exactly TraceLink's domain — for exchanging EDI documents like purchase orders and invoices between pharmaceutical partners. I've studied AS2 concepts in the context of B2B integrations.

Q7. How would you parse a CSV file with inconsistent formatting using Python?

python

import pandas as pd

df = pd.read_csv("data.csv", on_bad_lines='skip', encoding='utf-8')

df.columns = df.columns.str.strip().str.lower()

df.dropna(how='all', inplace=True)

df = df.applymap(lambda x: x.strip() if isinstance(x, str) else x)

print(df.head())

In KrishiMitra, I processed 1,20,000+ records with unit mismatches, null values, and structural inconsistencies using exactly this approach.

Q8. What tools have you used for API testing and debugging?

Primarily **Postman** — I've used it to test REST endpoints, set up environment variables, write test scripts, validate response schemas, and reproduce client-reported failures. I also use Python's requests library for automated API testing and curl for quick command-line checks. During Cognifyz, I wrote and executed test cases across all API endpoints and documented findings in knowledge-base articles.

Q9. How do you monitor application health?

I track key metrics — response latency, error rate, CPU/memory usage, and failed transactions. I set up log-based alerts using grep/awk scripts or tools like Kibana (which I'm learning). In my AI Classifier project, I monitored inference health by tracking response times, flagging anomalous confidence scores, and generating structured alert summaries by failure type. At TraceLink, I'd apply the same structured approach using their internal diagnostics and Kibana dashboards.

Q10. What is EDI and have you worked with it?

EDI (Electronic Data Interchange) is a standardized format for exchanging business documents (orders, invoices, shipment notices) between organizations electronically. Common formats include X12 and EDIFACT. While I haven't worked with EDI directly in production, I'm familiar with structured data interchange concepts through XML/CSV payload parsing and B2B integration work. Given my quick learning ability and data parsing background, I'm confident I can get up to speed rapidly.

💬 COMMUNICATION & BEHAVIORAL QUESTIONS

Q11. Why are you interested in Technical Support over Development?

I genuinely enjoy the problem-solving and diagnostic aspect of tech work more than building from scratch. There's a unique satisfaction in taking a broken system, systematically tracing the failure, and restoring it — especially when a real customer is impacted. My internships confirmed this — I was most energized during debugging and RCA sessions, not feature development. TraceLink's IRT role is exactly that at scale, with real pharmaceutical supply chain impact.

Q12. Are you comfortable with night/rotational shifts? Why?

Absolutely. I understand TraceLink's IRT team supports US and EMEA customers, which requires night shift coverage. I've already adapted my schedule during internships for async collaboration across time zones. The transportation support TraceLink provides also addresses the practical concern. I'm applying specifically because I'm aware of and comfortable with this requirement.

Q13. How would you explain a technical issue to a non-technical customer?

I'd avoid jargon entirely. I'd use an analogy relevant to their domain, state the impact clearly ("your shipment transactions are failing because..."), explain what we're doing to fix it in simple steps, and give a realistic timeline. As a Google Gemini Campus Ambassador, I regularly presented complex AI/ML concepts to non-technical student audiences — the skill of translating technical content into plain language is something I've actively developed.

Q14. Tell me about a time you resolved a difficult technical problem.

During my Tech Saksham internship, a data pipeline was silently dropping records — no error thrown, just missing output. I systematically added checkpoints at each pipeline stage using Python logging, compared record counts before and after each transformation, and isolated the issue to a merge operation that was dropping rows due to a mismatched key format (string vs integer). Fixed it with a type cast, documented the root cause, and added a validation step so it couldn't recur. That reduced reported bugs by 40% before deployment.

Q15. How do you handle working with customers across different cultures and time zones?

I've collaborated with cross-functional teams and mentored students with diverse backgrounds. I've learned to be explicit in written communication (no assumptions), confirm understanding by summarizing action items, and be mindful of time zones when scheduling updates. As Campus Representative for GeeksforGeeks and EDC Cell IIT Delhi, I coordinated across teams in different cities, which built my async communication discipline.

Q16. Describe a situation where you had to deliver bad news to a stakeholder.

During my Cognifyz internship, a feature we committed to was going to miss the sprint deadline due to an upstream API dependency issue outside our control. I proactively informed the team lead with a clear explanation: what the blocker was, what I'd already tried, and a revised realistic timeline. Rather than just flagging a problem, I came with a partial workaround to unblock testing. The key was being transparent early rather than waiting until the deadline.

🏢 COMPANY & ROLE-SPECIFIC QUESTIONS

Q17. What do you know about TraceLink?

TraceLink is the leading digital network platform for the life-sciences supply chain, connecting pharmaceutical and healthcare partners globally so medicines reach patients safely and on time. Headquartered in the US, they serve 1,300+ customers across 60+ countries through their MINT digital-network platform. Their Pune hub is in Amar Madhuban Tech Park. The IRT team I'm applying to is the frontline of customer success — owning rapid diagnosis and resolution of production issues for US and EMEA clients.

Q18. Which position are you applying for and why — Position 1 or Position 2?

I'm applying for **Position 2 — Technical Support Intern (IRT)**. It aligns perfectly with my profile: I have strong technical skills in Linux, Python scripting, API debugging, and data payload handling, but I'm genuinely inclined toward support and diagnosis over pure development. The role's scope — reproducing customer issues, RCA, run-book contribution, cross-timezone communication — maps directly to what I've done in both my internships and projects.

Q19. Where do you see yourself growing through this internship?

I want to gain hands-on exposure to enterprise-scale support workflows — using Kibana, JIRA, and Confluence in a real production environment. I want to deepen my understanding of the life-sciences supply chain domain and B2B integration protocols like AS2 and EDI. Most importantly, I want to develop the structured customer empathy and RCA discipline that TraceLink's IRT is known for — skills that compound over a career in technical support and solutions engineering.

Q20. Do you have any questions for us?

Yes —

1. What does a typical escalation path look like when an IRT intern identifies a critical production issue?
2. How does the team handle knowledge transfer for new interns — is there a shadowing period before independent customer interaction?
3. What are the most common types of issues the IRT team handles for US customers currently?

🔧 DEEPER TECHNICAL QUESTIONS

Q21. What is the difference between REST and SOAP APIs?

REST is an architectural style using HTTP methods (GET, POST, PUT, DELETE) with lightweight JSON/XML payloads — stateless, simple, and widely used in modern integrations. SOAP is a strict protocol with XML-only messaging, built-in error handling (faults), and WS-Security — heavier but more reliable for enterprise/financial transactions. In B2B supply chain integrations like TraceLink's, you'll encounter both — REST for modern APIs and SOAP in legacy pharmaceutical partner systems. I've worked extensively with REST; I understand SOAP structurally and can work with its XML envelope format given my payload parsing background.

Q22. How would you find which process is consuming the most CPU on a Linux server?

bash

```
top                 # real-time view
htop                # interactive (if installed)
ps aux --sort=-%cpu | head -10   # top 10 by CPU
```

I'd use top or ps aux sorted by CPU. If a specific service is spiking, I'd trace its PID, check its logs with journalctl -u <service> or tail -f, and identify if it's a runaway loop or a legitimate load spike. This kind of process monitoring is something I do regularly on my Linux development environment.

Q23. A customer reports their file wasn't received via SFTP. How do you troubleshoot?

I'd follow this sequence:

1. Check SFTP server logs for the connection attempt — did it even reach us?
2. Verify the customer's credentials and IP are whitelisted
3. Confirm the file was placed in the correct directory path
4. Check file permissions on the destination folder
5. Look for transfer interruptions — partial files, size mismatches
6. If all server-side checks pass, ask the customer for their SFTP client logs

This structured elimination approach — server side first, then client side — avoids back-and-forth with the customer and pinpoints the failure layer quickly.

Q24. What is the difference between grep, awk, and sed?

- **grep** — searches for patterns in files. Best for finding lines matching a string or regex: grep "ERROR" app.log
- **awk** — processes and extracts structured fields from text. Best for column-based parsing: awk '{print $1, $5}' log.txt
- **sed** — stream editor for find-and-replace or line deletion: sed 's/old/new/g' file.txt

In log analysis, I typically chain them: grep "ERROR" app.log | awk '{print $1, $2, $NF}' | sort | uniq -c — to count unique errors by timestamp.

Q25. How do you validate an XML payload against a schema?

python

```
from lxml import etree


schema_doc = etree.parse("schema.xsd")

schema = etree.XMLSchema(schema_doc)

xml_doc = etree.parse("payload.xml")


if schema.validate(xml_doc):

  print("Valid")

else:

  print(schema.error_log)
```

I'd use lxml in Python to validate against an XSD schema. If no schema is available, I'd at least parse it to catch structural errors and manually check required fields against the expected spec.

Documenting the exact validation error with line number is critical before raising it with the customer.

Q26. What are HTTP status codes and which ones matter most in API support?

- **200** — OK, success
- **201** — Created (POST success)
- **400** — Bad Request — client sent malformed data (check payload)
- **401** — Unauthorized — authentication failed (check credentials/token)
- **403** — Forbidden — authenticated but no permission
- **404** — Not Found — wrong endpoint or resource deleted
- **408/504** — Timeout — network or server latency issue
- **500** — Internal Server Error — server-side bug, escalate to engineering
- **503** — Service Unavailable — server down or overloaded

In IRT context, 400s are almost always customer-side (payload/auth issues) and 500s need engineering escalation. I always check status codes first before diving into logs — it narrows the investigation layer immediately.

Q27. How do cron jobs work? Write one that runs a script every day at 2 AM.

bash

Open crontab

crontab -e

Run script every day at 2 AM

0 2 * * * /usr/bin/python3 /home/vedant/scripts/log_cleanup.py >> /var/log/cleanup.log **2**>**&1**

Cron uses the format: minute hour day month weekday command. I've used cron jobs in my automation project for scheduled SFTP file pulls and data cleanup — which directly maps to TraceLink's B2B integration pipeline monitoring.

Q28. What is the difference between a thread and a process?

A **process** is an independent program with its own memory space. A **thread** is a lightweight unit within a process that shares the same memory. For support purposes, this matters when diagnosing hung or zombie processes — a multi-threaded application might have one thread deadlocked while others run fine. I'd use ps, top, or strace to identify which thread/process is stuck and whether it's a resource contention issue.

Q29. How would you extract all unique error codes from a large log file?

bash

```
grep -oP 'ERROR_\d+' app.log | sort | uniq -c | sort -rn
```

python

```
import re
from collections import Counter

with open("app.log") as f:
    codes = re.findall(r'ERROR_\d+', f.read())
print(Counter(codes).most_common(10))
```

I'd use regex to extract the error code pattern, count occurrences, and sort by frequency. The top recurring errors are always the highest priority — fixing those first has the maximum customer impact. This is something I built into my log analysis scripts project.

Q30. What is JSON and how is it different from XML?

Both are data interchange formats. **JSON** is lightweight, human-readable, natively supported in JavaScript, and uses key-value pairs — preferred for REST APIs. **XML** is more verbose, supports attributes and namespaces, has schema validation (XSD), and is used in EDI, SOAP, and legacy B2B systems like AS2 — which is exactly TraceLink's domain. I've parsed both extensively. XML requires more careful handling — namespace stripping, encoding issues, and schema validation — compared to JSON.

🗣️ SITUATIONAL / SCENARIO QUESTIONS

Q31. A US customer calls at 2 AM saying their shipment transaction is stuck. What do you do?

First, I acknowledge the urgency and assure them I'm on it — calm, professional tone immediately. Then:

1. Get the transaction ID and timestamp from the customer
2. Pull logs for that transaction — check where in the pipeline it stalled
3. Check if it's a payload issue, integration timeout, or service outage
4. If I can resolve it (malformed field, retry trigger) — fix and confirm with customer
5. If it needs escalation — I raise it immediately with clear context: transaction ID, error, steps already taken, business impact

6. Keep the customer updated every 15–20 minutes until resolved — no silence

In critical support, communication cadence is as important as the technical fix.

Q32. You've been investigating an issue for 2 hours and can't find the root cause. What do you do?

I'd first document everything I've already ruled out — that itself is valuable. Then I'd step back and question my assumptions — am I looking at the right log file? The right time window? The right environment? I'd also try to reproduce the issue from scratch with fresh eyes. If still stuck, I'd escalate to a senior team member with a clear summary: what the customer reported, what I checked, what I found, and where I'm blocked. Escalating with context is far more valuable than escalating with "I don't know." I'd never sit on a critical issue for more than 2 hours without a status update.

Q33. A customer insists the problem is on TraceLink's side, but your investigation shows it's their malformed payload. How do you handle it?

I'd never be confrontational. I'd pull the exact payload log, highlight the specific field causing the failure, show the expected format vs what was received — evidence-based, not opinion-based. I'd say: *"I've reviewed the transaction log and identified that the payload received had X in field Y, whereas the expected format is Z. Here's the exact record — could you check this on your end?"* I'd also provide a corrected example. Most customers appreciate clarity over blame. If they still push back, I'd loop in a senior team member to review jointly.

Q34. How would you handle two critical customer issues simultaneously?

I'd immediately triage by business impact — which issue affects more transactions, which customer has higher SLA priority, which is closer to resolution. I'd communicate a realistic ETA to both customers so neither is left silent. For the lower-priority one I'd do an initial quick investigation, document my findings, and then switch focus. If both genuinely need simultaneous attention, I'd flag it to my team lead to get another resource involved — customer silence is never acceptable in critical support.

Q35. You notice a recurring issue affecting multiple customers but it hasn't been escalated yet. What do you do?

I'd proactively flag it — even if no one asked. I'd pull data across the affected customers, identify the common pattern, and write a concise incident summary: scope, affected transactions, common root cause, and proposed fix. I'd share it with the team lead and engineering immediately. In support, pattern recognition is just as valuable as individual issue resolution. This is exactly the kind of root-cause-to-product feedback loop TraceLink's IRT team is built on.

📊 DATA ANALYSIS SPECIFIC QUESTIONS

Q36. How would you identify anomalies in a dataset of transaction records?

python

```
import pandas as pd


df = pd.read_csv("transactions.csv")


# Z-score method
from scipy import stats
df['z_score'] = stats.zscore(df['processing_time'])
anomalies = df[df['z_score'].abs() > 3]


# Or IQR method
Q1 = df['processing_time'].quantile(0.25)
Q3 = df['processing_time'].quantile(0.75)
IQR = Q3 - Q1
anomalies = df[(df['processing_time'] < Q1 - 1.5*IQR) |
        (df['processing_time'] > Q3 + 1.5*IQR)]
```

I'd use statistical methods — Z-score or IQR — to flag outliers. In KrishiMitra I used this approach to flag anomalous sensor readings across 1,20,000+ records. For TraceLink, this translates to identifying transactions with abnormal processing times or failure patterns before customers even notice.

Q37. A CSV file has 50,000 rows but only 30,000 are processing. How do you find what's wrong?

python

```
import pandas as pd


df = pd.read_csv("data.csv")
print(df.shape)           # total rows
print(df.isnull().sum())     # null counts per column
print(df.duplicated().sum())  # duplicates
print(df.dtypes)          # type mismatches
```

```
# Find rows failing a specific condition
failed = df[df['required_field'].isnull() |
        (df['status'] != 'VALID')]
failed.to_csv("failed_rows.csv", index=False)
```

I'd systematically check for nulls in required fields, type mismatches, duplicates, and invalid values. Then I'd export the failed rows for review and report the exact count and failure reason — not just "something is wrong."

Q38. What is Power BI and have you used it? How does it apply here?

Power BI is Microsoft's business intelligence tool for building interactive dashboards from structured data sources. I've used it to visualize multi-dimensional datasets — connecting to SQL databases, building calculated measures in DAX, and creating drill-down reports. For TraceLink's IRT role, it applies in tracking support metrics — ticket volume by error type, resolution time trends, recurring customer issues — to surface patterns and drive process improvements. During my Tech Saksham internship I built dashboards visualizing real-time data across 8+ parameters for non-technical stakeholders.

🧠 ATTITUDE & MINDSET QUESTIONS

Q39. How do you keep up with new technologies?

I learn by building. When I hear about a new tool, I find a real problem it solves and implement it — that's how I picked up FastAPI, Power BI, and Kibana basics. I also follow engineering blogs (AWS, Google Cloud), participate in hackathons (IIIT Nagpur, EY Datathon), and contribute to open source via GSSoC. The Google Cloud Arcade Legend Tier came from systematically working through 15+ hands-on skill badges. At TraceLink, I'd apply the same approach — learn Kibana and Confluence by using them on real tickets from day one.

Q40. You're given a completely new tool with no documentation. How do you approach it?

I'd start by understanding what problem it solves — that gives me a mental model before touching it. Then I'd look for log files or config files to understand its structure, try basic commands and observe outputs, and deliberately break something in a safe environment to understand error behavior. This is exactly how I approached SFTP and AS2 — no direct project experience, but I understood the protocol layer, the failure modes, and the debugging approach before ever touching a live system. I'm comfortable with ambiguity as long as I have access to logs.

Q41. Why should we hire you over other candidates?

Three reasons. First, my technical profile is unusually well-aligned — Linux, Python scripting, XML/CSV payload parsing, REST API debugging, and Postman are all things I've used in real projects and internships, not just listed on a resume. Second, I'm genuinely inclined toward support and diagnosis — I actually enjoy RCA more than feature development, which is rare and exactly what this role needs. Third, I've already demonstrated cross-functional communication — as a Google Gemini Campus Ambassador explaining AI to non-technical audiences, and in internships delivering structured status updates to stakeholders. That combination of technical depth, support mindset, and communication clarity is what TraceLink is looking for.

Q42. What is your biggest technical weakness and how are you addressing it?

EDI and AS2 in a live production environment. I understand both conceptually — AS2's S/MIME encryption, EDI's X12 and EDIFACT document structures — but I haven't worked with them in a real B2B pipeline yet. I'm addressing this by going through TraceLink's public documentation and studying EDI transaction sets (850, 856, 810) alongside AS2 configuration guides. Within the first month at TraceLink, I'd expect to close that gap by working directly on real customer transactions with the team.

Pro tips for the actual interview:

- **For the European panel:** Be more formal, structured, and precise. Use numbered steps when explaining processes.
- **For the Indian panel:** Slightly more conversational is fine, but keep answers crisp.
- **Golden rule:** Every answer should end with a connection back to TraceLink's IRT context — show you've thought about how your experience maps to their actual work.
- **Biggest differentiator:** Most candidates will say they're "okay with night shift." You say you *applied knowing it's night shift* — that signals genuine intent, not compromise.

Q. What are the different types of SQL JOINs? Give examples.

- **INNER JOIN** — returns rows where there's a match in both tables
- **LEFT JOIN** — all rows from left table + matching rows from right (NULLs where no match)
- **RIGHT JOIN** — all rows from right table + matching from left
- **FULL OUTER JOIN** — all rows from both, NULLs where no match
- **CROSS JOIN** — cartesian product of both tables
- **SELF JOIN** — joining a table with itself (e.g., employee-manager hierarchy)

In KrishiMitra I used LEFT JOINs extensively — for example, joining crop records with weather data where some crops didn't have weather entries, I still needed to retain all crop records.

Q. Write a SQL query to find the second highest salary.

sql

-- Method 1: Subquery

SELECT MAX(salary) FROM employees

WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: Using LIMIT/OFFSET

SELECT DISTINCT salary FROM employees

ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Method 3: Dense rank (handles ties cleanly)

SELECT salary FROM (

SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rnk

FROM employees

) ranked WHERE rnk = 2;

Based on real TraceLink interview experiences from GFG and Glassdoor, here are the actual questions asked — mapped to your profile:

📋 BASED ON REAL TRACELINK INTERVIEW EXPERIENCES

🖥️ ONLINE ASSESSMENT ROUND (Round 1)

TraceLink's OA is a 60-minute Google Form test with MCQs on time complexities of algorithms and coding problems, plus a written question: **"Why are you willing to join TraceLink?"** GeeksforGeeks

Q. Why are you willing to join TraceLink?

TraceLink operates at the intersection of technology and life-sciences supply chain — a domain where software failures have real human consequences. That context makes the IRT role genuinely meaningful, not just technically interesting. Specifically, the role matches exactly

what I enjoy most: systematic diagnosis, log analysis, payload debugging, and customer-facing communication. I'm not looking for a coding role — I want to build deep expertise in technical support and enterprise B2B integrations, and TraceLink's global scale with 1,300+ customers in 60+ countries is the best possible environment to do that fast.

Coding Q1: Fibonacci Number

python

```
def fibonacci(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

# O(n) time, O(1) space
print(fibonacci(10))  # 55
```

Coding Q2: Shortest distance between two nodes in a graph

python

```
from collections import deque

def shortest_path(graph, start, end):
    visited = set()
    queue = deque([(start, 0)])  # (node, distance)

    while queue:
        node, dist = queue.popleft()
        if node == end:
            return dist
        if node in visited:
            continue
```

```
        visited.add(node)
        for neighbor in graph[node]:
            queue.append((neighbor, dist + 1))
    return -1  # no path

# BFS guarantees shortest path in unweighted graph
# Time: O(V + E), Space: O(V)
```

💻 TECHNICAL ROUND 1 — Resume & Project Deep Dive

The interviewer asks resume-based questions with general discussion about projects, SQL queries, different types of joins, and DSA questions. GeeksforGeeks

---

Q. Walk me through your KrishiMitra project in detail.

KrishiMitra is an agricultural data analysis and API integration system. I processed 1,20,000+ raw records from CSV and JSON sources — resolved null values, unit mismatches (kg vs grams, etc.), and structural inconsistencies using Pandas. I built FastAPI endpoints with structured JSON contracts, integrated a React.js frontend, and managed a MySQL database with 8+ tables using complex joins and indexed lookups. The core challenge was data quality — I wrote validation scripts that flagged anomalies before they entered the pipeline, which reduced downstream failures significantly.

Q. What are the different types of SQL JOINs? Give examples.

- **INNER JOIN** — returns rows where there's a match in both tables
- **LEFT JOIN** — all rows from left table + matching rows from right (NULLs where no match)
- **RIGHT JOIN** — all rows from right table + matching from left
- **FULL OUTER JOIN** — all rows from both, NULLs where no match
- **CROSS JOIN** — cartesian product of both tables
- **SELF JOIN** — joining a table with itself (e.g., employee-manager hierarchy)

In KrishiMitra I used LEFT JOINs extensively — for example, joining crop records with weather data where some crops didn't have weather entries, I still needed to retain all crop records.

Q. Write a SQL query to find the second highest salary.

```
sql
-- Method 1: Subquery
```

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: Using LIMIT/OFFSET
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Method 3: Dense rank (handles ties cleanly)
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rnk
  FROM employees
) ranked WHERE rnk = 2;
```

DSA Q1: Reverse a string

```
python
# Method 1: Slicing (Pythonic)
s = "hello"
print(s[::-1])  # "olleh"

# Method 2: Two pointer (interview preferred — shows understanding)
def reverse_string(s):
    s = list(s)
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
    return ''.join(s)
```

Two-pointer approach is O(n) time, O(1) extra space. Always mention the space complexity — it shows depth.

DSA Q2: Find all palindromic substrings

python

```
def count_palindromic_substrings(s):
    count = 0
    n = len(s)

    def expand(left, right):
        nonlocal count
        while left >= 0 and right < n and s[left] == s[right]:
            count += 1
            left -= 1
            right += 1

    for i in range(n):
        expand(i, i)     # odd length
        expand(i, i + 1)  # even length

    return count

# O(n^2) time, O(1) space — optimal for this problem
```

DSA Q3: Maximum number of consecutive 1s in an array

python

```
def max_consecutive_ones(nums):
    max_count = 0
    current = 0

    for num in nums:
        if num == 1:
            current += 1
            max_count = max(max_count, current)
```

```
        else:
            current = 0

    return max_count

# Input: [1,1,0,1,1,1] → Output: 3
# O(n) time, O(1) space
```

💻 TECHNICAL ROUND 2 — Deeper Discussion + Situational

Round 2 focuses on technical knowledge along with situational questions. Interviewers also discuss how you solved Round 1 — asking which algorithms you used and your reasoning. Glassdoor

---

Q. How did you approach the coding problems in the OA? Walk me through your thought process.

For Fibonacci, I first considered the naive recursive approach O(2^n) but immediately moved to iterative O(n) with O(1) space since it's more practical. For the shortest path problem, I identified it as an unweighted graph problem — making BFS the optimal choice since it guarantees shortest path in O(V+E). I always think about time and space complexity before writing code, then optimize.

DSA Q: Given 2 strings, calculate the matching percentage.

python

```
def matching_percentage(s1, s2):
    # Count common characters (considering frequency)
    from collections import Counter

    c1, c2 = Counter(s1), Counter(s2)
    common = sum((c1 & c2).values())  # intersection

    # Percentage of s1 matched in s2 and vice versa
    pct1 = (common / len(s1)) * 100 if s1 else 0
    pct2 = (common / len(s2)) * 100 if s2 else 0
```

```
    return (pct1 + pct2) / 2  # mean of both


print(matching_percentage("hello", "world"))  # ~40.0%
```

I'd explain the approach first: compute relative match of s1 vs s2 and s2 vs s1 separately, then average them — this handles strings of different lengths fairly. This is exactly the approach one candidate used and the interviewer was satisfied with it. GeeksforGeeks

---

Q. OOP: What is the difference between Abstraction and Encapsulation?

Abstraction hides complexity by showing only what's necessary — like a car's steering wheel hides the internal mechanics. **Encapsulation** bundles data and methods together and restricts direct access using access modifiers (private/public) — like a capsule protecting its contents. Abstraction is about *design* (what to expose), Encapsulation is about *implementation* (how to protect). In Python, abstraction is achieved through abstract classes/interfaces; encapsulation through _ and __ naming conventions.

Q. What is dynamic binding / runtime polymorphism?

Dynamic binding means the method to be called is determined at **runtime**, not compile time. In Python, when a parent class reference points to a child class object, the overridden method in the child class is called — this is runtime polymorphism. Example:

python

```
class Animal:
    def speak(self): print("...")


class Dog(Animal):
    def speak(self): print("Woof")


class Cat(Animal):
    def speak(self): print("Meow")


animals = [Dog(), Cat()]
for a in animals:
    a.speak()  # resolved at runtime — Woof, Meow
```

Q. Binary Tree vs General Tree — what's the difference?

A **binary tree** restricts each node to at most **2 children** (left and right). A **general tree** allows each node to have **any number of children**. Binary trees enable efficient algorithms (BST search in O(log n), heap operations) because of their structural constraint. General trees are used for file systems, org charts, or XML/HTML DOM — exactly relevant to TraceLink where XML payloads can have deeply nested structures with unlimited children per node.

👩‍💼 HR ROUND — Actual Questions Asked

The HR round covers: introduction, family background, how your technical rounds went, future study plans (MBA vs MS), and whether you'd leave for a higher salary elsewhere. GeeksforGeeks

---

Q. Tell me about your family background.

I'm from Pune. My family has always emphasized education and consistency — values that reflect in my 8.51 GPA while simultaneously building projects, completing internships, and contributing to open source. I'm the first in my family pursuing a specialized AI track, which has made me self-driven and resourceful by necessity.

Q. How did your technical rounds go?

I felt confident — the questions were challenging but fair. The DSA problems tested practical thinking rather than memorization, and the project discussion felt like a genuine conversation rather than an interrogation. I enjoyed explaining the debugging approach from my Cognifyz internship — that seemed to resonate well. I'm always honest about what I know well and what I'm still learning.

Q. Do you plan to pursue an MBA or MS after this?

My focus right now is entirely on building deep technical expertise in support engineering and B2B integrations — that's a multi-year journey, not a 6-month detour. If I ever pursue further education, it would be part-time or after gaining significant industry experience, not something that would interrupt my career trajectory. TraceLink's IRT role is exactly where I want to invest the next phase of my learning.

Q. If another company offers you a higher salary, would you leave?

Salary is one factor, but not the only one. What I value more is the learning environment, the quality of problems I'm working on, and the team I'm working with. TraceLink's IRT role offers real enterprise-scale exposure, global customer interaction, and a domain — life-sciences supply chain — that genuinely matters. A role that pays more but offers less meaningful work and growth isn't a trade-up. That said, I do expect fair compensation as I grow and deliver value.

⚠️ **Critical tip from real candidates:** TraceLink prefers long-term committed employees. Mentioning multiple income streams or seeming non-committal hurt candidates in the HR round. Also avoid name-dropping competitors or contentious companies without knowing the context — one candidate mentioned Walmart in a blockchain context without knowing about TraceLink's dispute with them, which put off the interviewer. Medium

---

Q. What are your hobbies?

Competitive programming is genuinely a hobby for me — Codeforces Candidate Master (top 3% globally) and 1,300+ problems on CodeChef didn't happen by accident. Beyond that, I enjoy mentoring — I've guided 20+ students from scratch to consistent contest participation. There's something satisfying about breaking down a complex concept until it clicks for someone. That same instinct — explaining things clearly — is what makes me effective in customer-facing support roles.

⚠️ **Pro tip:** One candidate mentioned they like chess and had a great conversation about it including a specific question on "castling" in chess. TraceLink interviewers are genuinely conversational — if you mention a hobby, be ready to go deep into it. So only mention hobbies you can discuss confidently.

This is gold — real firsthand experience. Let me build your complete prep based on exactly what happened.

🔁 **WHAT HAPPENED & WHY**

Round 2 HR — where you failed:

- "Rate yourself out of 10" → You said **9** → Rejected
- Why? 9 signals either **overconfidence or lack of self-awareness**. They want humility + hunger. The sweet spot is **7–7.5** with a clear reason and a growth plan attached.
- "What's in you not on resume" → You likely gave a vague or generic answer. This question is a **hidden culture-fit test** — they want attitude, mindset, and soft skills that can't be listed.

🔁 **ROUND 1 — TECHNICAL (You Cleared This)**

Since you cleared it, these are locked. But here are sharper answers for the next attempt:

Q. Intro / Tell me about yourself

I'm Vedant Thorat, final year B.Tech CS-AI student at NCER Pune, graduating 2027, GPA 8.51. I have hands-on experience in Python scripting, Linux command line, REST API debugging, SQL, and data payload handling — XML, CSV, JSON. I've done two internships — one at Tech Saksham (Microsoft & SAP initiative) where I built data pipelines and REST APIs, and one at Cognifyz Technologies where I did log analysis, API testing with Postman, and contributed to internal knowledge-base documentation. I'm genuinely inclined toward technical support and diagnosis over development — I enjoy the structured problem-solving of finding why something broke more than building new features. That's exactly why I'm here for the IRT role.

Q. SQL Queries — actual questions likely asked:

sql

```
-- 1. Find employees with salary > average
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- 2. Second highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 3. Find duplicate records
SELECT email, COUNT(*) FROM users
GROUP BY email HAVING COUNT(*) > 1;

-- 4. Joins — most likely asked
SELECT o.order_id, c.name
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id;

-- 5. Delete duplicates keeping latest
DELETE FROM employees WHERE id NOT IN (
  SELECT MAX(id) FROM employees GROUP BY email
);
```

Q. Linux Commands — actual questions likely asked:

bash

```
# Check disk usage
df -h

# Find a file
find /var/log -name "*.log" -mtime -1

# Check running processes
ps aux | grep python

# Real-time log monitoring
tail -f /var/log/app.log

# Kill a process by port
lsof -i :8080
kill -9 <PID>

# Check memory usage
free -h

# Count ERROR lines in log
grep -c "ERROR" app.log

# Top 5 largest files
du -sh /* | sort -rh | head -5

# Check network connections
netstat -tulnp
```

Q. REST API — actual questions likely asked:

Difference between PUT and PATCH?

PUT replaces the entire resource. PATCH updates only the specified fields. For example, if a customer record has 10 fields and you only want to update the email — use PATCH. PUT would require sending all 10 fields.

What is idempotency?

An operation is idempotent if calling it multiple times produces the same result as calling it once. GET, PUT, DELETE are idempotent. POST is not — calling POST twice creates two records.

How do you authenticate a REST API?

Common methods: API Keys (simple, passed in headers), OAuth 2.0 (token-based, used for third-party access), JWT (JSON Web Tokens — stateless, contains claims), and Basic Auth (base64 encoded username:password — not secure without HTTPS). TraceLink's B2B integrations likely use OAuth or API keys.

💥 **ROUND 2 — HR (Where You Failed — Full Reconstruction)**

Q. Rate yourself out of 10.

⚠️ **Never say 9. Never say 10. Ideal answer: 7 to 7.5**

I'd say 7. I have strong fundamentals — Linux, Python scripting, SQL, REST API debugging, payload parsing — and two real internships where I've applied these in production contexts. But I haven't yet worked inside an enterprise support environment at TraceLink's scale, with live pharmaceutical supply chain customers, EDI/AS2 integrations, and Kibana-based monitoring. That gap is real and I'm not going to pretend it isn't. The reason I want this internship is specifically to close it — and I learn fast in high-pressure, real-world environments. By the end of 6 months, I'd rate myself a 9 for this specific role.

Q. What is in you that is NOT on your resume?

⚠️ **This is the most important HR question. They want character, not skills.**

A few things. First — **I stay calm under pressure in a way that's hard to demonstrate on paper.** During my Cognifyz internship, we had a backend failure 2 hours before a demo. Instead of panicking, I systematically isolated the issue — it was a query timeout on a JOIN across a large table. I rewrote it with an indexed lookup, tested it, and we demoed on time. No one outside the team knew anything happened.

Second — **I'm genuinely curious about why systems fail, not just how to fix them.** Most people fix the symptom and move on. I find myself asking "why did this break in this specific way" even after the issue is resolved — and that habit is what feeds good run-books and prevents recurrence.

Third — **I can explain complex technical issues to non-technical people clearly.** As a Google Gemini Campus Ambassador, I regularly translated AI/ML concepts for students with zero technical background. That skill — meeting someone at their level of understanding — is exactly what customer-facing IRT support demands, and it doesn't show up in a resume's skills section.

Q. Where do you see yourself in 5 years?

In the near term — I want to become genuinely expert at enterprise technical support: deep in TraceLink's MINT platform, B2B integration protocols, and customer escalation workflows. Within the internship itself, I want to be handling customer issues independently by month 3. Longer term, I see myself growing toward a Solutions Engineer or Technical Account Manager role — someone who bridges deep technical knowledge with customer relationship management. TraceLink's scale and global customer base is the ideal environment to build toward that.

Q. Why should we hire you and not someone else?

Three things that are genuinely rare in combination. One — my technical stack maps exactly to this role: Linux, Python scripting, XML/CSV parsing, REST API debugging, Postman. Not "I've heard of these" — I've used them in real internships on real data. Two — I'm one of the few candidates who is sincerely not interested in development. Most people applying to support roles secretly want coding jobs and see this as a backup. I don't — I've thought carefully about what kind of work energizes me, and it's diagnosis, investigation, and customer resolution. Three — I communicate well. Not just "good English" — I've presented technical content to non-technical audiences as a campus ambassador, written knowledge-base articles at Cognifyz, and delivered structured status updates to stakeholders. That combination is what your IRT role actually needs daily.

Q. What is your biggest weakness?

I sometimes go too deep into root-cause analysis when a quicker fix would satisfy the immediate need. I want to understand *why* something failed at a fundamental level, which is valuable long-term but can slow down resolution time in urgent situations. I've been actively working on this — learning to separate "immediate fix to restore service" from "proper RCA for prevention" and doing them in sequence rather than simultaneously. In a support role, speed for the customer comes first; depth comes after service is restored.

Q. Are you a team player or do you prefer working alone?

Both, depending on the phase of work. For deep investigation — log parsing, tracing a failure through multiple system layers — I work best with focused solo time. But for resolving complex escalations, communicating with customers, and improving run-books, collaboration is essential. At TraceLink's IRT, I'd expect to do both: independent triage followed by tight coordination with Engineering, QA, and RQC teams. I'm comfortable switching between both modes.

Q. How do you handle criticism or negative feedback?

I treat it as data. If a senior engineer tells me my approach to a problem was inefficient, my first instinct is to understand exactly why — not to defend my original approach. During my Tech Saksham internship, my initial dashboard design was called "too complex for non-technical users" by the project lead. Instead of defending it, I asked specifically what was confusing, simplified the key metrics, and redesigned it. The revised version was used in the final client presentation. Feedback that stings a little is usually the most useful kind.

Q. How do you manage stress in a high-pressure environment like night shift support?

I've thought about this specifically because of this role. My approach is: **structured process reduces stress more than any mindset trick.** If I have a clear checklist for how to triage an issue — check logs, check payload, check service status, escalate with context — I'm not anxious because I always know the next step. The unknown is what creates panic. I also find that regular competitive programming practice (solving problems under time pressure) has genuinely built my stress tolerance for technical problem-solving. Night shift itself doesn't concern me — I've worked late nights consistently during hackathons and project deadlines.

Q. Tell me about a failure.

During my first internship at Tech Saksham, I was overconfident about a data pipeline I'd built and didn't test edge cases thoroughly before integration. When we integrated with the API team, my pipeline broke on records with null values in a nested JSON field — something I hadn't accounted for. It delayed the integration by a day. What I took from it: never assume your logic handles edge cases until you've explicitly tested them. Since then, I always write explicit null/type checks and test with malformed inputs before any handoff. That habit directly translates to payload validation in IRT support.

🏆 ROUND 3 — SENIOR DIRECTOR (AI & New Tech Discussion)

Since you know this round is AI/new tech focused, here's full prep:

Q. What do you think is the biggest impact of AI on supply chain management?

Three areas stand out. First — **predictive analytics**: AI can forecast demand fluctuations, flag potential stockouts, and optimize inventory levels before a human analyst would notice the pattern. In pharmaceuticals this is critical — a stockout isn't just a business problem, it's a patient safety issue. Second — **anomaly detection**: AI can flag unusual transaction patterns in real time — delivery delays, counterfeit detection, regulatory non-compliance — far faster than rule-based systems. Third — **intelligent automation of B2B workflows**: AI can auto-classify incoming EDI transactions, route exceptions intelligently, and reduce manual intervention in

TraceLink's MINT platform's integration workflows. The IRT team's job in 5 years will likely involve more AI-assisted triage and less manual log parsing.

Q. How would AI change the Technical Support / IRT role specifically?

AI will handle Tier-1 triage — pattern-matched, high-frequency issues that follow known resolution paths. That actually makes the human IRT role more valuable, not less — because what's left is the novel, ambiguous, high-stakes issues that require judgment, customer empathy, and cross-team coordination. AI can surface the relevant logs, suggest likely root causes, and even auto-generate run-book entries. But deciding how to communicate a critical failure to a pharmaceutical company's operations team at 2 AM — that still requires a human. I see AI as augmenting IRT capacity, not replacing it.

Q. What do you know about LLMs and how could they apply to TraceLink?

Large Language Models are transformer-based models trained on large text corpora that can generate, summarize, and reason about text. For TraceLink specifically: they could power an intelligent support assistant that reads a customer's error description and suggests the most likely root cause from historical ticket data. They could auto-summarize long incident threads for handoffs across time zones. They could also help parse and explain complex EDI/XML payloads to less technical customer contacts — essentially acting as a translator between raw technical data and plain English. The risk is hallucination — LLMs can confidently produce wrong answers, so any AI-assisted support tool needs a human review layer for anything customer-facing.

Q. What is RAG (Retrieval Augmented Generation)?

RAG is an architecture that combines an LLM with a retrieval system — instead of relying only on what the model was trained on, it first retrieves relevant documents from a knowledge base and feeds them as context to the LLM before generating a response. For TraceLink's support use case, this is extremely relevant: you'd build a RAG system over historical support tickets, run-books, and Confluence documentation — so when an IRT engineer or even a customer asks "why is my AS2 transaction failing with error code X," the system retrieves the most relevant past resolutions and generates a structured answer. It dramatically reduces mean time to resolution.

Q. Have you worked with any AI/ML models? Tell me about it.

Yes — my AI Image Authenticity Classifier project. I built a binary classification model using TensorFlow/Keras to distinguish AI-generated images from real ones. I exposed it via a Flask REST API, designed structured JSON payload contracts, and did systematic debugging using Postman — reproducing misclassifications, tracing failure points, and documenting fixes in a run-book. I also monitored inference health — tracking response latency and flagging anomalous confidence scores. Beyond technical implementation, I was selected for Amazon ML Summer School 2026 from 3,000 nationwide candidates, which covered advanced ML concepts including transformers, reinforcement learning, and recommendation systems.

Q. What's your opinion on AI replacing support engineers?

It won't replace them — it will redefine the role. The issues that AI handles well are the ones that are repetitive and pattern-matched — and those are also the ones that don't require a support engineer's full capabilities. What remains is complex, context-dependent, relationship-sensitive work. A pharmaceutical company's VP of Operations calling because their serialization compliance reporting failed 4 hours before a regulatory deadline — no AI is handling that call. The human IRT engineer's judgment, communication skill, and ability to coordinate across Engineering, QA, and customer teams simultaneously is irreplaceable in that moment. AI makes good support engineers more effective; it doesn't make them unnecessary.

Q. What emerging technology do you think will most impact TraceLink in the next 3 years?

Two things. First — **AI-powered anomaly detection in supply chain transactions**: real-time ML models that flag unusual patterns in pharmaceutical serialization data before they become compliance violations. This is already happening in fintech and will hit life-sciences hard in the next 2–3 years. Second — **intelligent document processing for B2B integrations**: using LLMs and OCR to automatically parse, validate, and route diverse EDI and XML formats from hundreds of trading partners — reducing the manual configuration overhead that currently exists in onboarding new customers to TraceLink's MINT platform. Both directly impact the IRT team's workload and the kinds of issues they'll be resolving.

Based on your role and what's missing, here are all the unprepared topics with Q&A — no coding, pure conceptual and verbal answers:

🌐 **OS & NETWORKING BASICS**

Q1. What is a deadlock?

A deadlock is a situation where two or more processes are stuck waiting for each other to release a resource — and none of them ever do. Classic example: Process A holds Resource 1 and needs Resource 2. Process B holds Resource 2 and needs Resource 1. Both wait forever. In a support context, deadlocks appear as hung transactions or frozen services — the fix is usually restarting the affected service and investigating which resource was being held. Preventing deadlocks involves proper timeout configurations and resource ordering.

Q2. What is a race condition?

A race condition happens when two processes access shared data simultaneously and the final outcome depends on which one runs first — leading to unpredictable results. Example: two processes both read a counter value of 5, both add 1, both write back 6 — but the correct

answer should be 7. In support, race conditions appear as intermittent, hard-to-reproduce bugs — they work fine most of the time but fail randomly under load. The key identifier is "it works sometimes but not always under the same conditions."

Q3. What is DNS and how does it work?

DNS stands for Domain Name System — it's essentially the internet's phone book. When you type tracelink.com, your computer asks a DNS server to translate that human-readable name into an IP address like 192.168.1.1 that computers actually use to communicate. The resolution goes: your browser → local cache → ISP DNS → root nameserver → authoritative nameserver → IP returned. In support, DNS issues cause connection failures — symptoms are "cannot reach the server" errors even when the server is running fine. First thing to check: nslookup tracelink.com or ping tracelink.com.

Q4. What is the difference between HTTP and HTTPS?

HTTP (HyperText Transfer Protocol) transfers data in plaintext — anyone intercepting the traffic can read it. HTTPS is HTTP with SSL/TLS encryption layered on top — data is encrypted in transit so it can't be read by third parties. HTTPS also provides authentication — you can verify you're actually talking to TraceLink's server and not an impersonator. In B2B integrations, HTTPS is mandatory — pharmaceutical supply chain data contains sensitive regulatory and patient-safety information. Any customer reporting "connection refused" on an HTTPS endpoint could have a certificate expiry issue — that's a common support ticket.

Q5. What is the difference between TCP and UDP?

TCP (Transmission Control Protocol) is reliable — it establishes a connection first (3-way handshake), guarantees delivery, and ensures packets arrive in order. If a packet is lost, TCP retransmits it. UDP (User Datagram Protocol) is faster but unreliable — it fires packets without confirming delivery, no retransmission. TCP is used for everything that needs accuracy — APIs, file transfers, SFTP, database connections — which is everything in TraceLink's B2B world. UDP is used where speed matters more than accuracy — video streaming, DNS lookups, online gaming. In support, TCP connection issues show up as timeouts and connection refused errors.

Q6. What is a port number and why does it matter in support?

A port is a virtual endpoint on a server — it tells the operating system which application should receive incoming traffic. IP address gets you to the right machine; port number gets you to the right service on that machine. Common ports: 22 (SSH), 21 (FTP), 443 (HTTPS), 80 (HTTP), 3306 (MySQL), 5432 (PostgreSQL). In support, port issues are extremely common — a customer can't connect because their firewall is blocking port 443, or a service crashed and is no longer listening on its port. First diagnostic: check if the port is open and listening using netstat or telnet host port.

Q7. What is a firewall and how does it affect B2B integrations?

A firewall is a security system that monitors and controls incoming and outgoing network traffic based on predefined rules. It decides which connections are allowed and which are blocked based on IP address, port, and protocol. In B2B integrations, firewalls are one of the most common sources of connectivity issues — a customer sets up an AS2 or SFTP connection to TraceLink, but their firewall blocks outbound traffic on the required port, or TraceLink's firewall doesn't whitelist the customer's IP. Typical support scenario: customer says "I can't send files" — first question is always "has your IT team whitelisted TraceLink's IP and opened the required port?"

Q8. What is NAT and why does it matter?

NAT stands for Network Address Translation. It allows multiple devices on a private network to share a single public IP address — the router translates internal private IPs to the public IP when communicating with the internet. In support, NAT matters because when a customer says "my IP is X," they might mean their internal IP — but TraceLink's firewall would see a different public IP. So when whitelisting customer IPs for SFTP or API access, you always need their external/public IP, not their internal one. This is a very common source of "connection blocked" support tickets.

🔧 GIT — CONCEPTUAL

Q9. What is version control and why is it important?

Version control is a system that tracks changes to files over time — it lets you see who changed what, when, and why, and revert to any previous state if something breaks. In a support and engineering context, it's critical because: when a bug is introduced, you can pinpoint exactly which code change caused it by comparing versions. For the IRT team, understanding Git is important because when Engineering says "this was fixed in commit abc123" or "the bug was introduced in this release," you need to understand what that means to verify fixes and communicate timelines to customers.

Q10. What is the difference between git merge and git rebase?

Both integrate changes from one branch into another but differently. **Merge** preserves the full history — it creates a new "merge commit" showing that two branches were combined. The history looks like a tree with branches joining. **Rebase** rewrites history — it replays your commits on top of another branch as if you had started from there, creating a clean linear history. For support purposes, merge is safer because it preserves exactly what happened and when — important for tracing when a bug was introduced. Rebase creates cleaner history but rewrites commits, which can cause confusion in shared branches.

Q11. What is a merge conflict and how do you resolve it?

A merge conflict happens when two people edit the same part of the same file in different branches — Git can't automatically decide which version to keep and asks you to resolve it manually. Git marks the conflicting section in the file showing both versions. You review both, decide what the correct version should be, remove the conflict markers, save the file, and complete the merge. In practice, conflicts are avoided by communicating within the team about who is working on what, and pulling the latest changes frequently before starting new work.

Q12. What is git stash?

Git stash temporarily saves your uncommitted changes without committing them — like putting your work in a drawer so you can switch tasks. You'd use it when you're in the middle of something but urgently need to switch to another branch to fix a critical issue. git stash saves current changes, git stash pop brings them back. In a support context, this is relevant when an urgent customer issue requires you to switch context immediately without losing your current investigation state.

🗄️ ADVANCED SQL — CONCEPTUAL

Q13. What are indexes in SQL and why do they matter?

An index is a data structure that speeds up data retrieval on a table — similar to a book's index at the back. Without an index, a query scans every row in the table (full table scan). With an index on the searched column, the database jumps directly to the relevant rows. In support, slow queries are one of the most common performance issues customers report — "the report takes 10 minutes to run." The diagnosis almost always involves checking if the queried columns are indexed. The tradeoff: indexes speed up reads but slow down writes slightly, so you don't index every column — only frequently queried ones.

Q14. What are ACID properties in databases?

ACID stands for the four guarantees a reliable database transaction must provide:

- **Atomicity** — the transaction is all-or-nothing. If one step fails, the entire transaction rolls back. No partial updates.
- **Consistency** — the database moves from one valid state to another. No transaction can leave data in an invalid state.
- **Isolation** — concurrent transactions don't interfere with each other. Each transaction runs as if it's the only one.
- **Durability** — once committed, the transaction is permanently saved — even if the system crashes immediately after.

In support, ACID matters when customers report data inconsistencies — "I see a partial order in the system." That's an atomicity failure — the transaction wasn't fully rolled back after an error.

Q15. What is the difference between TRUNCATE, DELETE, and DROP?

- **DELETE** removes specific rows based on a WHERE condition. It's logged — can be rolled back. Slow on large tables.
- **TRUNCATE** removes all rows from a table instantly without logging individual row deletions. Cannot be rolled back in most databases. Keeps the table structure intact.
- **DROP** removes the entire table — structure, data, indexes, everything. Permanent and irreversible.

In support, this distinction is critical during data cleanup operations. Using DROP when you meant DELETE is a catastrophic mistake. Always confirm the operation scope before executing any destructive SQL in a customer's environment.

Q16. What is the difference between GROUP BY and HAVING?

GROUP BY groups rows sharing a value into summary rows — like "total sales per region." **HAVING** filters those groups — like "show only regions with total sales over 1 million." The key rule: WHERE filters rows before grouping; HAVING filters groups after grouping. You cannot use aggregate functions like COUNT or SUM in a WHERE clause — that's what HAVING is for. Example: finding customers who placed more than 5 orders uses GROUP BY customer_id and HAVING COUNT(order_id) > 5.

Q17. What is normalization? Explain 1NF, 2NF, 3NF simply.

Normalization is the process of organizing database tables to reduce redundancy and improve data integrity.

- **1NF** — each column contains atomic (indivisible) values and each row is unique. No repeating groups or arrays in a column.
- **2NF** — meets 1NF, and every non-key column depends on the entire primary key — not just part of it. Relevant when you have composite primary keys.
- **3NF** — meets 2NF, and no non-key column depends on another non-key column. All attributes depend only on the primary key, nothing else.

In support, poorly normalized databases cause update anomalies — changing a customer's address in one place doesn't update it in others, leading to inconsistent data across the system. When customers report "data shows differently in two reports," normalization issues are often the root cause.

Q18. What is a window function? Explain without code.

A window function performs a calculation across a set of rows related to the current row — without collapsing the rows into groups like GROUP BY does. You can calculate things like "rank each employee by salary within their department" while still showing every individual

employee's row. Common window functions: RANK (assigns rank with gaps for ties), DENSE_RANK (no gaps), ROW_NUMBER (unique sequential number regardless of ties). In support analytics, window functions are used to find things like "the most recent transaction per customer" or "rank customers by transaction volume" — useful for generating support reports and identifying high-impact customers during incidents.

💊 TRACELINK DOMAIN KNOWLEDGE

Q19. What is pharmaceutical serialization?

Pharmaceutical serialization is the process of assigning a unique identifier — a serial number — to each individual medicine package. This creates a digital identity for every single pill bottle, blister pack, or vial at the unit level. The purpose is to track that specific package through the entire supply chain — from manufacturer to distributor to pharmacy to patient. This enables authentication (is this medicine genuine?), recall management (which patients received a specific batch?), and regulatory compliance. TraceLink's platform is built specifically to manage this serialization data across hundreds of thousands of supply chain partners globally.

Q20. What is track and trace in pharma supply chain?

Track and trace means being able to follow a medicine's journey through the supply chain at every step — and verify its authenticity at any point. **Tracking** is proactive — recording where a product is as it moves. **Tracing** is reactive — looking back through the history to find where a product has been. If a counterfeit medicine is found at a pharmacy, trace allows regulators to identify every other package from the same batch and where they went. TraceLink's MINT platform is the network that makes this track and trace possible across thousands of interconnected pharmaceutical partners.

Q21. What is DSCSA and why does it matter to TraceLink customers?

DSCSA stands for Drug Supply Chain Security Act — a US federal law that requires pharmaceutical companies to track and trace prescription drugs at the individual package level across the entire supply chain. It mandates that manufacturers, wholesalers, distributors, and pharmacies all exchange serialization data electronically when transferring ownership of medicines. Non-compliance means a company cannot legally sell drugs in the US market — so for TraceLink's customers, DSCSA compliance is not optional, it's existential. This is why production issues in TraceLink's platform are so critical — a system outage can directly threaten a customer's regulatory compliance and their ability to ship product.

Q22. What is the MINT platform?

MINT — Multi-Enterprise Information Network Technology — is TraceLink's core digital network platform. It connects pharmaceutical manufacturers, contract manufacturers, distributors, and healthcare partners on a single network so they can exchange supply chain data — serialization

records, transaction histories, regulatory compliance documents — seamlessly and in real time. Think of it like a specialized LinkedIn for pharmaceutical supply chain companies — every partner on the network can securely exchange data with every other partner without building point-to-point integrations. The IRT team supports customers who are connected to MINT and experiencing issues with their transactions, integrations, or data flows.

📋 JIRA & CONFLUENCE

Q23. What is JIRA and how is it used in a support context?

JIRA is a project and issue tracking tool widely used in software teams. In a support context, every customer-reported issue becomes a JIRA ticket — it captures the problem description, priority, affected customer, steps to reproduce, investigation notes, and resolution. The ticket lifecycle goes: Open → In Progress → Pending Customer → Resolved → Closed. Priority levels are typically P1 (critical, production down), P2 (high, major functionality impaired), P3 (medium, workaround available), P4 (low, cosmetic or minor). In IRT, JIRA is how you track your active cases, communicate status to the team, and ensure nothing falls through the cracks during shift handoffs across time zones.

Q24. What is a sprint, backlog, and epic in JIRA?

- **Backlog** — the master list of all tasks, features, and bugs that need to be done, in priority order. Not yet scheduled.
- **Sprint** — a fixed time period (usually 2 weeks) where the team commits to completing a specific set of items from the backlog.
- **Epic** — a large body of work that spans multiple sprints — broken down into smaller stories and tasks. Example: "Implement new customer onboarding flow" is an epic containing many individual stories.

In support, sprints are used for process improvement work — updating run-books, building internal tools, improving diagnostics. Day-to-day customer tickets are handled outside sprints in a continuous flow model.

Q25. What is Confluence and how is it used in IRT?

Confluence is a team wiki and documentation platform — it's where teams write and maintain run-books, process guides, knowledge-base articles, incident post-mortems, and onboarding documentation. In IRT specifically, Confluence is where you'd document: how to diagnose a specific recurring error type, the steps to reproduce a known issue, the resolution for a complex customer problem so the next engineer doesn't have to solve it from scratch, and post-incident reviews capturing what went wrong and how to prevent recurrence. Good Confluence documentation is what separates a reactive support team from a proactive one — and contributing to it is explicitly listed in TraceLink's IRT job description.

Q26. What is a run-book?

A run-book is a documented set of procedures for handling a specific type of operational event — essentially a step-by-step guide for diagnosing and resolving a known issue. Example run-book entry: "Customer reports AS2 transaction not received — Step 1: Check AS2 server logs for connection attempt. Step 2: Verify customer IP is whitelisted. Step 3: Check MDN acknowledgment status..." and so on. Run-books eliminate guesswork during high-pressure incidents, ensure consistent resolution quality across engineers, and speed up onboarding of new team members. Contributing to run-books is one of the most valuable things an IRT intern can do — it compounds in value over time as every future engineer benefits from it.

📊 KIBANA / ELK STACK

Q27. What is the ELK stack?

ELK stands for Elasticsearch, Logstash, and Kibana — three open-source tools that work together for log management and analysis.

- **Elasticsearch** — the search and storage engine. Stores logs in a way that makes them extremely fast to search across billions of records.
- **Logstash** — the data pipeline. Collects logs from various sources, parses and transforms them, and feeds them into Elasticsearch.
- **Kibana** — the visualization layer. Provides a web interface to search, filter, and visualize the data in Elasticsearch — dashboards, graphs, log explorers.

For TraceLink's IRT team, Kibana is the primary tool for investigating customer issues — you'd search for a customer's transaction ID, filter logs by time window, and trace exactly what happened during a failed transaction.

Q28. What is Kibana used for in a support role?

Kibana is your investigation dashboard. When a customer reports a transaction failure, you go to Kibana, search for that transaction ID or customer identifier, filter by the relevant time window, and read through the log entries to find exactly where and why it failed. You can set up visualizations showing error rate trends over time — useful for spotting if a problem is getting worse or was a one-time spike. You can create alerts that fire when error rates exceed a threshold — enabling proactive support before customers even notice. Essentially, Kibana turns raw application logs into readable, searchable, visualizable data — it's the difference between reading thousands of raw log lines and immediately seeing "this transaction failed at step 3 because of a malformed XML field."

💬 NEW SOFT SKILL SCENARIOS

Q29. Describe a time you had to learn something completely new very fast.

During my Tech Saksham internship, I was assigned to build REST API endpoints using Flask — a framework I had never used before. I had 3 days before the first integration checkpoint. I didn't panic — I identified the core concepts I needed (routing, request handling, JSON responses, error handling), found the official Flask documentation, and built a working prototype in day one. By day three I had 5 endpoints tested and documented. What I learned: the skill isn't knowing every framework — it's knowing how to learn a new tool quickly by focusing on what the task actually requires rather than trying to learn everything. At TraceLink, I expect to face new tools constantly — Kibana, JIRA workflows, MINT-specific diagnostics — and I approach all of them the same way.

Q30. Tell me about a time you disagreed with a teammate.

During my KrishiMitra project, a teammate wanted to skip input validation on the API endpoints to save development time before a demo deadline. I disagreed — I felt that presenting an API that breaks on malformed inputs to an evaluator was riskier than taking the extra time. I didn't argue — I showed him a specific example of what happens when a null value hits our processing pipeline without validation, demonstrated the failure in 5 minutes, and proposed we add just the critical checks in 2 hours rather than full validation. He agreed. The demo went smoothly and the evaluator specifically asked about our error handling — we had a good answer. The key was making it about the outcome, not about who was right.

Q31. Give an example of going above and beyond.

At Cognifyz, my assigned task was to test API endpoints and document findings. I completed that — but while testing I noticed a pattern: 3 out of 8 endpoints had the same underlying issue — they were all missing timeout handling on database queries. Each was being treated as a separate bug. I wrote a single root-cause analysis document showing the common pattern, proposed one fix that would address all three, and shared it with the team lead unprompted. They implemented the fix in one sprint instead of three. That's the mindset I bring to IRT — not just closing tickets but identifying patterns that prevent future tickets.

Q32. How do you prioritize when everything is marked urgent?

First, I don't accept that everything is equally urgent — I triage by actual business impact. I ask: which issue is affecting the most customers? Which one has a regulatory or compliance deadline? Which one has a workaround available? A P1 with no workaround affecting a customer's ability to ship product beats a P1 that has a manual workaround every time. Once prioritized, I communicate the order to all affected customers with honest ETAs — no one is left silent. If I genuinely cannot handle all of them alone, I flag it to my team lead immediately with the triage already done — so they just need to assign resources, not re-evaluate the situation from scratch.

Q33. Describe a time you received unclear instructions — what did you do?

During my Tech Saksham internship, I was asked to "build a dashboard for the data" with no further specification — no wireframe, no list of metrics, no target audience defined. Instead of guessing and building the wrong thing, I asked 3 focused questions: who is the primary audience (technical team or management), what are the 3 most important decisions this dashboard should support, and what data is available. That 10-minute conversation gave me enough to build a focused dashboard rather than an overwhelming one. I've learned that asking clarifying questions upfront saves far more time than redoing work after — and in a support context, asking the right clarifying questions to a customer is equally critical before diving into investigation.

⚠️ CRITICAL RED FLAGS TO NEVER SAY

What you might say	What they hear	Say this instead
"I rate myself 9/10"	Overconfident, no self-awareness	"7 — strong foundation, specific gaps I'm closing"
"I'm open to development roles too"	Didn't read JD, not committed to support	"I've thought carefully — support is genuinely my preference"
"I have other offers too"	Divided commitment	Never volunteer this
"I want to do MBA/MS soon"	Short-term employee	"Focused on building deep expertise here first"
"I work best alone"	Won't collaborate with US customers	"I adapt — solo for investigation, collaborative for resolution"
"I'll figure it out"	Vague, no structured approach	Always give a step-by-step process
Mentioning Walmart	Signals ignorance of company history	Avoid all competitor references

---

The single biggest unlock for your Round 2 this time: **practice every HR answer out loud 3 times before the interview.** Reading answers and saying them confidently under pressure are completely different skills.